

Branch and price for the 3D-SCPTOP

and what “optimal” does and does not mean in our results

31 August 2026

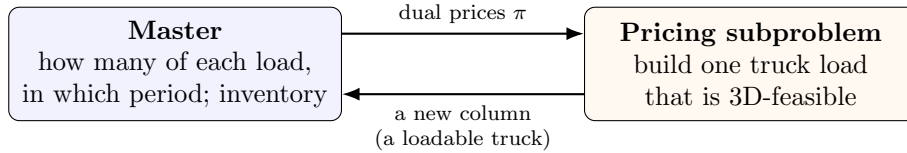
Abstract

This note explains the branch-and-price method as implemented, and states precisely what our results guarantee. The short answer to the second question: **we do not in general prove optimality, and we should not claim to**. What we do guarantee is different and, for this problem, arguably more valuable — **every truck we report is proven loadable**. The distinction matters because the software once reported a solution as `OPTIMAL` that its own lower bound admitted could be 24% above optimal.

1 Why decompose at all

The compact 3D-SCPTOP carries a variable for every combination of truck, loading configuration, product and period. Two consequences follow. It is enormous — 68 904 variables on a four-product, ten-period instance — and it is *symmetric*: trucks are interchangeable, so the branch-and-bound tree explores permutations of the same plan. On the ten-period instances Gurobi returns no feasible solution at all within half an hour.

Dantzig–Wolfe decomposition moves the geometry out of the model and into the *columns*. A column is one truck’s load: a specific set of returnable transport items, placed in specific slots, that we have already checked can physically be built. The master problem then never reasons about packing. It only chooses *how many of each load to dispatch, and when*.



2 The two problems

Master (restricted). Let Ω be the set of all geometrically feasible truck loads — far too many to enumerate — and $\Omega' \subset \Omega$ the ones we have generated so far. For a load k , a_{ck} is how many units of commodity c it carries, and c_K is the cost of dispatching a truck. With $y_{kt} \geq 0$ the number of copies of load k dispatched in period t , and I the inventory variables:

$$\min c_K \sum_{k \in \Omega'} \sum_t y_{kt} + (\text{inventory cost}) \quad \text{s.t.} \quad \sum_{k \in \Omega'} a_{ck} y_{kt} + (\text{flow, stock}) \geq d_{ct}$$

This is a linear program. Solving it gives an objective value z_{RMP} and dual prices π_{ct} on the demand rows.

Pricing. The duals say what a unit of each commodity is worth. We then ask: *is there a loadable truck worth more than it costs?* That is a 3D packing problem solved with CP-SAT, subject to every physical rule — slot placement, stack crush limits, trailer payload, and the full-truckload rule (a truck must be full *by volume or by weight*, not by their sum):

$$\max_{a \in \Omega} \sum_c \pi_c a_c \quad \implies \quad \text{reduced cost} = c_K - \sum_c \pi_c a_c.$$

If the reduced cost is negative, that load improves the master, and we add it and re-solve. If *no* load has negative reduced cost, the restricted master is optimal for the *full* problem Ω — and this is the only circumstance under which we learn anything certain.

One implementation detail that is not cosmetic. Dual prices are scaled to integers for CP-SAT and **rounded up, never to nearest**. Rounding to nearest can make the oracle believe a column is unprofitable when it is not, which destroys the proof in the next section; convergence would then never be declared.

3 Bounds: the part that decides whether we may say “optimal”

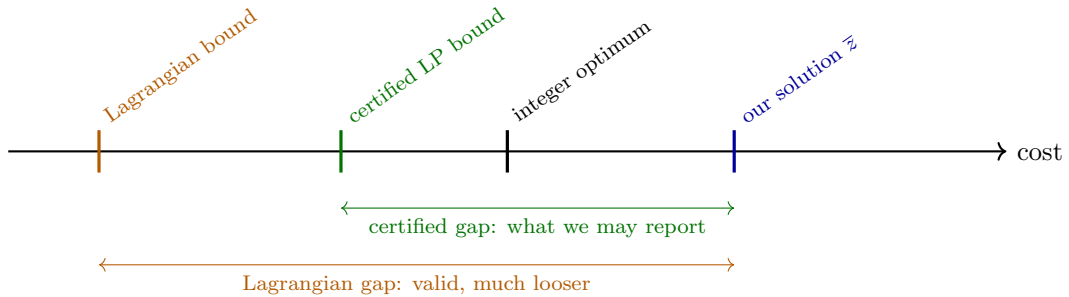
This is where the reasoning is easy to get wrong, and where we got it wrong.

The restricted master’s LP value is not a lower bound. z_{RMP} is computed over a *subset* of columns. Adding columns can only lower it. It is therefore an *upper* bound on the true master LP value z_{LP} .

Three situations, and what each licenses us to say:

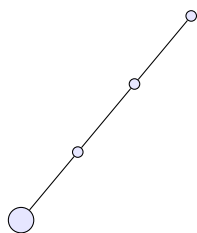
Situation	What is known	May we quote a gap?
Pricing proved no improving column	$z_{\text{RMP}} = z_{\text{LP}}$, a valid lower bound	yes, a <i>certified</i> gap
Pricing ran but did not close	Lagrangian bound applies	yes, a weaker valid gap
Pricing never ran (deadline)	nothing from below	no

The middle case uses the Lagrangian (Farley) correction: if ρ is the most negative reduced cost found and κ an upper bound on how many columns an optimal solution can use, then $z_{\text{LP}} \geq z_{\text{RMP}} + \kappa\rho$. It is valid but loose — across our benchmark it averages a 28.6% gap, which describes the bound, not the plans.



4 From fractional to integer

Column generation solves the *linear relaxation*. Integrality still has to be recovered, and here is the honest limitation of our method: we use **depth-first diving**, a heuristic. We repeatedly fix the most fractional variable and re-solve, and we do not explore the alternative branch. A full branch-and-price would; ours does not.



we follow one branch to the bottom (a “dive”)

the branches we never open are why no optimality proof follows from the dive

Consequently: **a dive can prove a solution exists, never that none is better**. Optimality can only come from the bound, never from the dive.

5 What our statuses actually mean

The software previously reported the *restricted master’s own* solver status. **OPTIMAL** there means “the integer master was solved to optimality over the columns that happened to be generated” — a statement about our column pool, not about the 3D-SCPTOP. Two symptoms made this concrete:

- Two runs of the same instance under identical settings both reported **OPTIMAL**, at 4250.54 and 4252.88. Two genuine optima of one problem cannot differ; the column pools differed.
- One stored result read **PLC:OPTIMAL** beside an objective of 4248.48 and its own lower bound of 3222.88 — a solution that might be 24% above optimal, labelled optimal.

Statuses now say which case holds:

Status	Meaning
(proven)	pricing closed <i>and</i> the gap is zero: genuinely optimal
(pool; gap≤X% vs certified bound)	within X% of a bound pricing proved
(pool; gap≤X% vs Lagrangian bound)	within X% of a valid but weaker bound
(pool; no valid bound)	a feasible plan; nothing known from below

6 The guarantee we *do* make

Optimality is the weaker claim here. The stronger one, and the point of the paper, is **feasibility**:

Every truck in every reported plan is re-verified independently of the model that produced it — payload, the full-truckload floor, per-RTI-type slot capacity, and stack crush limits.

This is not ceremony. The verifier caught three distinct classes of impossible plan during development: a truck loaded to 24 567 kg on a 24 000 kg trailer; a truck dispatched 1.4% full, far under the utilisation floor; and a configuration asked for 25 slots where 24 exist. None of these was caught by the optimisation model, which considered each of them perfectly feasible.

The contrast with the volumetric benchmark is the paper’s central result. SCPTOP treats a trailer as a volume, solves in *under one second*, and reports plans 13.7% cheaper than ours — of which, on the first benchmark instance, **18 of its 20 truck loads are proven impossible to pack**. A cheaper plan that cannot be loaded is not a cheaper plan.

7 Where this leaves the results

Claim	Status
“Our plans are loadable”	Guaranteed , verified truck by truck.
“Our plans are optimal”	Not claimed. True only where a status reads (proven).
“Within $X\%$ of optimal”	Claimed only against a valid bound, and the bound’s kind is named.
“Better than the compact model”	Claimed only at <i>equal measured wall-clock</i> ; an earlier version of this comparison gave the matheuristic 400 s against Gurobi’s 1800 s, and its conclusions were an artefact of that handicap.

Empirically, certification is rare at the budgets we use: the pricing oracle closed the subproblem on 1 of 18 runs. The strongest single result is the ten-period instance where the compact model finds nothing in half an hour and the matheuristic returns a plan with a *certified* gap of 1.42% — and there, “certified” carries its full meaning.